JENSON USA

Data Analysis

Business Problem Overview

Jenson USA is an e-commerce retailer specializing in bicycles and cycling accessories.
The company wants to understand its sales performance across stores and products to support better business decisions.

The key challenges are:

- Identifying which stores are selling the most products.
- Understanding top-selling products and categories.
- Analyzing sales trends over time.
- Evaluating revenue contribution by each store etc.

Using SQL, this analysis helps the business optimize inventory, improve store performance, and increase overall revenue.

1.Find the total number of products sold by each store along with the store name.

```sql
-- Find the total number of products sold by each store along with the store name
select s.store_name,sum(oi.quantity) as total_product_sold
from order_items oi
join orders o
on oi.order_id=o.order_id
join stores s
on o.store_id=s.store_id
group by s.store_name;
```

2.Calculate the cumulative sum of quantities sold for each product over time.

```sql
-- 2.Calculate the cumulative sum of quantities sold for each product over time.
SELECT
    p.product_name,
    YEAR(o.order_date) AS year,
    MONTH(o.order_date) AS month,
    SUM(oi.quantity) AS monthly_quantity,
    SUM(SUM(oi.quantity)) OVER (
        PARTITION BY p.product_name
        ORDER BY YEAR(o.order_date), MONTH(o.order_date)
    ) AS cumulative_quantity
FROM order_items oi
JOIN orders o
    ON oi.order_id = o.order_id
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY
    p.product_name,
    YEAR(o.order_date),
    MONTH(o.order_date)
ORDER BY
    p.product_name, year, month;
```

3.Find the product with the highest total sales (quantity * price) for each category.

```sql
-- Find the product with the highest total sales (quantity * price) for each category.
WITH product_sales AS (
    SELECT
        c.category_name,
        p.product_name,
        SUM(oi.quantity * oi.list_price) AS total_sales,
        RANK() OVER (
            PARTITION BY c.category_name
            ORDER BY SUM(oi.quantity * oi.list_price) DESC
        ) AS sales_rank
    FROM order_items oi
    JOIN products p
        ON oi.product_id = p.product_id
    JOIN categories c
        ON p.category_id = c.category_id
    GROUP BY
        c.category_name,
        p.product_name
)
SELECT
    category_name,
    product_name,
    total_sales
FROM product_sales
WHERE sales_rank = 1;
```

4.Find the customer who spent the most money on orders.

```sql
-- Find the customer who spent the most money on orders.
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(oi.quantity * oi.list_price) AS total_spent
FROM order_items oi
JOIN orders o
    ON oi.order_id = o.order_id
JOIN customers c
    ON o.customer_id = c.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
ORDER BY total_spent DESC
LIMIT 1;
```

5.Find the highest-priced product for each category name.

```sql
-- Find the highest-priced product for each category name.
SELECT
    c.category_name,
    p.product_name,
    p.list_price
FROM products p
JOIN categories c
    ON p.category_id = c.category_id
JOIN (
    SELECT
        category_id,
        MAX(list_price) AS max_price
    FROM products
    GROUP BY category_id
) mp
    ON p.category_id = mp.category_id
   AND p.list_price = mp.max_price;
```

6.Find the total number of orders placed by each customer per store.

```sql
-- Find the total number of orders placed by each customer per store
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    s.store_name,
    COUNT(o.order_id) AS total_orders
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN stores s
    ON o.store_id = s.store_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name,
    s.store_name
ORDER BY
    c.customer_id,
    s.store_name;
```

7.Find the names of staff members who have not made any sales.

```sql
-- Find the names of staff members who have not made any sales
SELECT
    s.staff_id,
    s.first_name,
    s.last_name
FROM staffs s
LEFT JOIN orders o
    ON s.staff_id = o.staff_id
WHERE o.staff_id IS NULL;
```

8.Find the top 3 most sold products in terms of quantity.

```sql
-- Find the top 3 most sold products in terms of quantity.
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity) AS total_quantity_sold
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY
    p.product_id,
    p.product_name
ORDER BY
    total_quantity_sold DESC
LIMIT 3;
```

9.Find the median value of the price list.

```sql
-- Find the median value of the price list.
SELECT AVG(list_price) AS median_price
FROM (
    SELECT
        list_price,
        ROW_NUMBER() OVER (ORDER BY list_price) AS rn,
        COUNT(*) OVER () AS cnt
    FROM products
) t
WHERE rn IN (
    FLOOR((cnt + 1) / 2),
    CEIL((cnt + 1) / 2)
);
```

10.List all products that have never been ordered.(use Exists).

```sql
-- List all products that have never been ordered.(use Exists)
SELECT
    p.product_id,
    p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);
```

11.List the names of staff members who have made more sales than the average number of sales by all staff members.

```sql
-- List the names of staff members who have made more sales than the average number of sales by all staff members
WITH staff_sales AS (
    SELECT
        s.staff_id,
        s.first_name,
        s.last_name,
        COUNT(o.order_id) AS total_sales
    FROM staffs s
    LEFT JOIN orders o
        ON s.staff_id = o.staff_id
    GROUP BY
        s.staff_id,
        s.first_name,
        s.last_name
),
avg_sales AS (
    SELECT AVG(total_sales) AS avg_sales
    FROM staff_sales
)
SELECT
    ss.first_name,
    ss.last_name,
    ss.total_sales
FROM staff_sales ss
CROSS JOIN avg_sales a
WHERE ss.total_sales > a.avg_sales;
```

12.Identify the customers who have ordered all types of products (i.e., from every category).

```sql
-- Identify the customers who have ordered all types of products (i.e., from every category)
SELECT
    c.customer_id,
    c.first_name,
    c.last_name
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY
    c.customer_id,
     c.first_name,
     c.last_name
 HAVING
     COUNT(DISTINCT p.category_id) = (
         SELECT COUNT(DISTINCT category_id)
         FROM categories
     );
```